Exercise 3

Alex WIlson
Introduction to Statistical Methods

January 31, 2021

Abstract

In the following report, two different hypotheses for the law of refraction will be tested, by minimising χ^2; the accuracy of each fit will be assessed, against a given set of data. To assess the 'goodness-of-fit' for each fit p-values will be calculated,using the chi2.sf function from the scipy.stats package, and the previously calculated χ^2_{min} values. Then a least squares fit will be generated for the known law of refraction, and again the χ^2_{min} value will be found.

1 Data

θ_i (degrees)	θ_r (degrees)
10.0	8.0 ± 0.5
20.0	15.5 ± 0.5
30.0	22.5 ± 0.5
40.0	29.0 ± 0.5
50.0	35.0 ± 0.5
60.0	40.5 ± 0.5
70.0	45.5 ± 0.5
80.0	50.0 ± 0.5

Table 1: Ptolemy's data set, given in the script[1],with angles of incidence θ_i, and angles of refraction, θ_r. The error on the angles of incidence is considered to be negligible.

2 Hypotheses

Least squares fits for the data in table 1, where generated using curve_fit , with hypotheses 1 and 2.

$$\theta_r = \alpha\theta_i \tag{1}$$

$$\theta_r = \alpha\theta_i - \beta\theta i^2 \tag{2}$$

These were coded into python using the following code:

```
def func1(i,*theta_r):
    alpha, = theta_r
    return alpha*i
p01 = np.ones(1)
thetahat1,cov1 = curve_fit(func1, i, r,p01 , sig,
    absolute_sigma=True)
def func2(i,*theta_r):
    alpha, beta = theta_r
    return alpha*i-beta*i**2
p02 = np.ones(2)
thetahat2,cov2 = curve_fit(func2, i, r,p02 , sig,
    absolute_sigma=True)
```

To give parameter values of $\alpha = 0.66 \pm 0.003$, for equation 1; $\alpha = 0.83 \pm 0.01$, and $\beta = 2.5 \pm 0.2{\cdot}10^{-3}$, for equation 2. The calculated χ^2_{min} values for the hypotheses were 134 for equation 1, giving a p-value of $6.7{\cdot}10^{-26}$ suggesting the hypothesis was not accurate for the given data points; and 0 for equation 2, giving a p-value of 1, meaning that all of the data points fitted the hypothesis perfectly, suggesting that the data points were not based on actual measurements, and were more likely generated from this hypothesis.

χ^2_{min} and p-values calculated using the following code:

```
chisq_list1=[]
chisq_list2=[]
for a in range(len(i)):
    chisq_list1.append(((r[a]-func1(i[a],*thetahat1)
        )/(sig[a]))**2)
    chisq_list2.append(((r[a]-func2(i[a],*thetahat2)
        )/(sig[a]))**2)
chisq1=sum(chisq_list1)
chisq2=sum(chisq_list2)
print('Chisq1',chisq1,'\nChisq2',chisq2)
print('\nChisq1/ndof',chisq1/8,'\nChisq2/ndof',
    chisq2/7)
print ("\np-value for equation 1 = {0}\n".format(
    chi2.sf(chisq1, df=7)))
print ("p-value for equation 2 = {0}\n".format(chi2.
    sf(chisq2, df=6)))
```

where the fit parameters:thetahat1, thetahat2,their standard deviations, sig, and the data points and used to make the individual terms in the χ^2 formula, then summed together giving the minimum χ^2 value. These minimum χ^2 values are then used along with the chi2.sf function to produce p-values for each, with the degrees of freedom, df, given by the number of parameters subracted from the number of measurements.

3 Plots

Plots of the hypotheses from equations 1 and 2, made using curve_fit and matplotlib.

4 Snell's Law

Snell's law is given by equation3

$$\theta_r = sin^{-1}\left(\frac{sin(\theta_i)}{r}\right) \tag{3}$$

where r is the ratio of refractive indices, of the two different materials. Using the code given below, the parameter r was calculated:

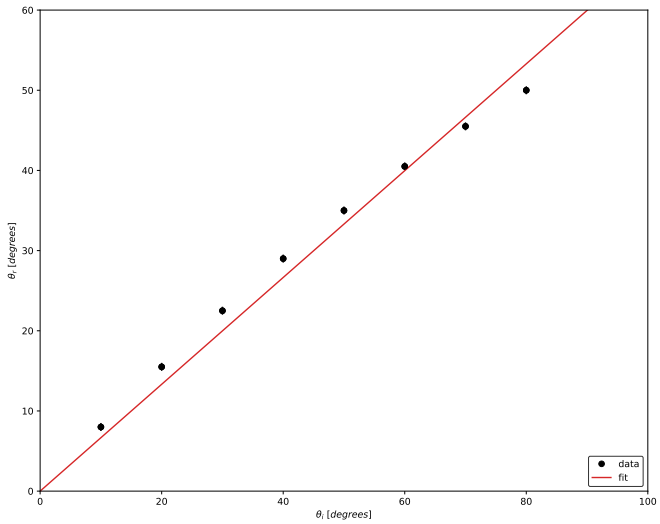

Figure 1: Plot of the angle of incidence, θ_i, against the angle of refraction, θ_i (both in degrees), with the linear fit from equation 1. As can be seen visually this hypothesis was not a good fit for this data.

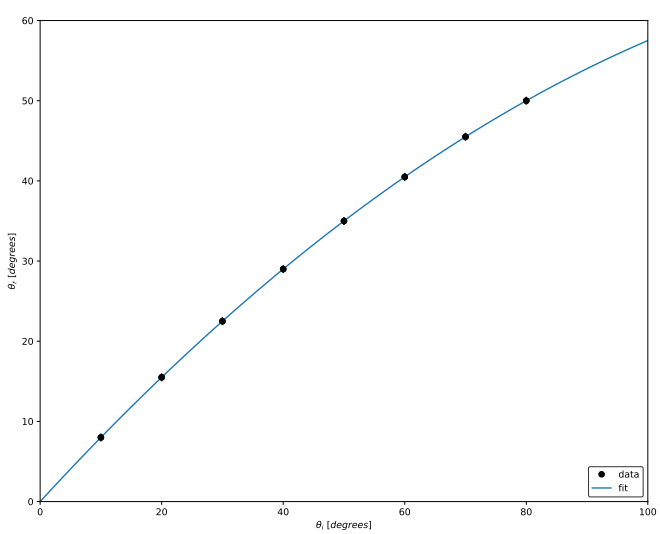

Figure 2: Plot of the angle of incidence, θ_i, against the angle of refraction, θ_i (both in degrees), with fit containing quadratic correction from equation 2. This fit goes through every data point perfectly, suggesting that the data is not based on actual measurements.

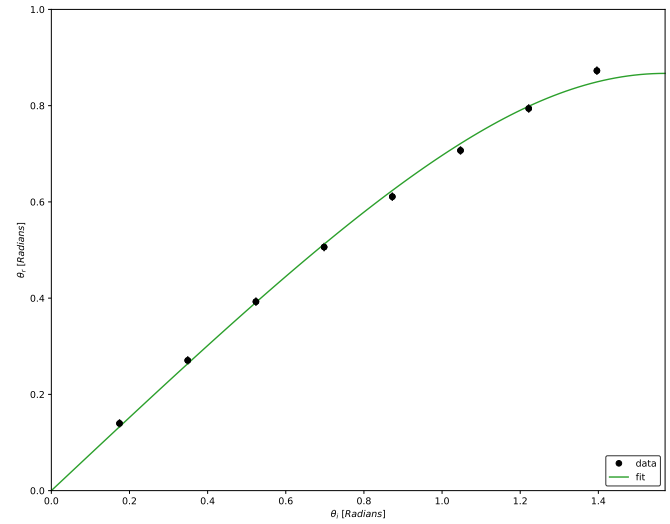

Figure 3: Plot of the angle of incidence, θ_i, against the angle of refraction, θ_i, this time in radians. The overlaid fit is the generated from the accepted law of refraction hypothesis given in 3, as it can be seen it does not fit the data points perfectly, this is likely again due to them not being based on actual measurements.

```
i_=np.deg2rad(i)
r_=np.deg2rad(r)
sig_=np.deg2rad(sig)
def func3(i_,*theta_r):
    alpha, = theta_r
    return np.arcsin(np.sin(i_)/(alpha))
p03 = np.ones(1)
thetahat3,cov3 = curve_fit(func3, i_, r_,p03 , sig,
    absolute_sigma=True)
```

where the angles from table 1 are converted from degrees to radians using the deg2rad from numpy. The fuction func3 takes inputs of the angles θ_i and a placeholder parameter *theta_r , and returns a function relating the two, that can then be used in curve_fit to produce a parameter that fits the relation between i_, and r_ to the function. Using this method a least-squares estimate for r was given as: r = 1,3 ± 0.3, with a χ^2_{min} of 14.0 giving a p-value of 0.05. These values might suggest that the assumed value of the standard deviation being: $\sigma = \frac{1}{2}^{\circ}$, may have been too small.

A Python Code

Code used to create parameters, plots and statistical data, shown below:

```
'''
==========
  Exercise 3
==========
'''

import matplotlib
import matplotlib.pyplot as plt
import numpy as np
from scipy.optimize import curve_fit
from scipy.stats import chi2
```

```python
'''
===
Data
===
'''
xmin=0
xmax=100
ymin=0
ymax=60
i   = np.arange(10, 90,step=10)
r   = np.array([8,15.5,22.5,29,35,40.5,45.5,50])
sig = np.ones(8)*0.5
x_= np.linspace(xmin,xmax,num=100)
'''
==========
Functions 1 & 2
==========
'''
def func1(i,*theta_r):
    '''
    Inputs: i, *theta_r
    * denotes vector
    Output: linear function with only one parameter,
        alpha
    '''
    alpha, = theta_r
    return alpha*i
p01 = np.ones(1)
thetahat1,cov1 = curve_fit(func1, i, r,p01 , sig,
    absolute_sigma=True)

def func2(i,*theta_r):
    alpha, beta = theta_r
    return alpha*i-beta*i**2
p02 = np.ones(2)
thetahat2,cov2 = curve_fit(func2, i, r,p02 , sig,
    absolute_sigma=True)

def StDev(x,cov,M):
    k=[]
    for i in range(M):
        for j in range(M):
            k.append(x**(i+j)*cov[i,j])
    return (np.sqrt(sum(k)))

'''
=======
Function 3
=======

'''
i_=np.deg2rad(i)
r_=np.deg2rad(r)
sig_=np.deg2rad(sig)
def func3(i_,*theta_r):
    alpha, = theta_r
    return np.arcsin(np.sin(i_)/(alpha))
p03 = np.ones(1)
thetahat3,cov3 = curve_fit(func3, i_, r_,p03 , sig,
    absolute_sigma=True)
'''
====
Tabel
====
'''
for o in range(len(i)):
    print('%.1f & %.1f \pm %.1f \\\\' % (i[o], r[o],
        sig[o]))
    '''
=====
Values
=====
'''
t_hats=[thetahat1,thetahat2,thetahat3]
covs=[cov1,cov2,cov3]
stdevs=[]
stdevs2=[]
print(covs)
for p in range(len(t_hats)):
    if p!=1:
        stdevs.append(np.sqrt(covs[p][0][0]))
        print('Parameter for equation {0}:\n  alpha
            = {1}+-{2}\n'.format(p+1,t_hats[p][0],
            stdevs[p]))
    elif p==1:
        stdevs.append(np.sqrt(covs[p][0][0]))
        stdevs2.append(np.sqrt(covs[p][1][1]))
        print('Parameters for equation {0}:\n  alpha
            = {1}+-{2}\n  beta = {3}+-{4}\n'.format
            (p+1,t_hats[p][0],stdevs[p],t_hats[p
            ][1],stdevs2[p-1]))
'''
====
Plot 1
====
'''
fig1 = plt.figure(figsize=(10, 8))
ax1 = fig1.add_subplot(1,1,1)

ax1.plot(i,r,'o',color='black',label='data')
ax1.errorbar(i,r,yerr=sig,fmt='o',color='black')
ax1.plot(x_,func1(x_,*thetahat1),color='tab:red',
    label='fit')

ax1.set_xlabel(r'$\theta_{i}$ $[degrees]$')
ax1.set_ylabel(r'$\theta_{r}$ $[degrees]$')

ax1.legend(frameon='true',edgecolor='black',loc='
    lower right')
ax1.set_xlim(xmin,xmax)
ax1.set_ylim(ymin,ymax)
plt.tight_layout()

fig1.savefig("Plot 1.pdf", format='pdf')

'''
====
Plot 2
====
'''

fig2 = plt.figure(figsize=(10, 8))
ax2 = fig2.add_subplot(1,1,1)

ax2.plot(i,r,'o',color='black',label='data')
ax2.errorbar(i,r,yerr=sig,fmt='o',color='black')
ax2.plot(x_,func2(x_,*thetahat2),color='tab:green',
    label='fit')

ax2.set_xlabel(r'$\theta_{i}$ $[degrees]$')
ax2.set_ylabel(r'$\theta_{r}$ $[degrees]$')

ax2.legend(frameon='true',edgecolor='black',loc='
    lower right')
ax2.set_xlim(xmin,xmax)
ax2.set_ylim(ymin,ymax)
plt.tight_layout()

fig2.savefig("Plot 2.pdf", format='pdf')

'''
====
Plot 3
====
'''
x=np.linspace(0,np.pi/2,num=100)
fig3 = plt.figure(figsize=(10, 8))
ax3 = fig3.add_subplot(1,1,1)

ax3.plot(i_,r_,'o',color='black',label='data')
ax3.errorbar(i_,r_,yerr=sig_,fmt='o',color='black')
ax3.plot(x,func3(x,*thetahat3),color='tab:green',
    label='fit')
```

```
ax3.set_xlabel(r'$\theta_{i}$ $[Radians]$')
ax3.set_ylabel(r'$\theta_{r}$ $[Radians]$')

ax3.legend(frameon='true',edgecolor='black',loc='lower right')
ax3.set_xlim(0,np.pi/2)
ax3.set_ylim(0,1)

plt.tight_layout()

fig3.savefig("Plot 3.pdf", format='pdf')

'''
=====
Goodness of fit
=====
'''
chisq_list1=[]
chisq_list2=[]
chisq_list3=[]
for a in range(len(i)):
    chisq_list1.append(((r[a]-func1(i[a],*thetahat1))/(sig[a]))**2)
    chisq_list2.append(((r[a]-func2(i[a],*thetahat2))/(sig[a]))**2)
    chisq_list3.append(((r_[a]-func3(i_[a],*thetahat3))/(sig_[a]))**2)
chisq1=sum(chisq_list1)
chisq2=sum(chisq_list2)
chisq3=sum(chisq_list3)
print('Chisq1',chisq1,'\nChisq2',chisq2)
print('\nChisq1/ndof',chisq1/8,'\nChisq2/ndof',chisq2/7)
print ("\np-value for equation 1 = {0}\n".format(chi2.sf(chisq1, df=7)))
print ("p-value for equation 2 = {0}\n".format(chi2.sf(chisq2, df=6)))
print ("p-value for equation 3 = {0}\n".format(chi2.sf(chisq3, df=7)))
```

Exercise_3.py

References

[1] G. Cowan. *Introduction to Statistic Methods*. Royal Holloway, University of London. 2021.